

Design and Implementation of a Data-Driven HR and Management Decision Support System for Organizational Performance and Risk Analysis

Mikhael Nabil Salama Rezk
IHUTSC

University Consultant: Mark Kovacs

Bachelor Thesis

**Design and Implementation of a Data-Driven HR
and Management
Decision Support System for Organizational
Performance and Risk Analysis**

Prepared by:

Mikhael Nabil Salama Rezk

Neptun Code: IHUTSC

Degree Program: Computer Science and Systems Engineering

Department: Department of Computer Science and Systems Engineering

University: John Von Neumann University

Supervisor: Mark Kovacs

Submission Year: 2025

Assignment Sheet

Insert the official university thesis or diploma assignment sheet on this page before final submission.

This placeholder is included to mirror the formal structure of the university Word template. Replace this page with the signed and approved assignment sheet required by your faculty.

Table of Contents

Introduction

1. Problem Definition and Objectives
2. Literature Review and Related Work
3. System Design and Architecture
4. Implementation and System Development
5. Data Processing Pipeline
6. Key Performance Indicators and Metrics
7. Role-Based Dashboard Design
8. Results and System Evaluation
9. Use Cases and Practical Applications
10. Limitations and Challenges
11. Future Work and Research Directions

Conclusions

Summary

List of Figures

References

Attachments

Appendix A. System Installation and Deployment Guide

Appendix B. Input Normalization Functions — Mathematical Specification

Appendix C. Test Case Specifications and Results

Appendix D. Code Architecture Overview

Appendix E. Future Implementation Checklist

Appendix F. Database Schema and Persistence Design

Appendix G. Route Catalog (Condensed)

Appendix H. Source-to-Section Traceability

Introduction

I started this prototype because I noticed a recurring frustration in how companies handle data: we're drowning in numbers but starving for actual clarity. Most HR software does a decent job of counting heads, but it's useless when you need to know if your leadership is actually ready for a scaling event or if a crisis is brewing under the surface.

My vision was to pull 'soft' people metrics and 'hard' financial data into a single, honest view. I intentionally avoided 'black box' AI models for the primary engine. Instead, I built a transparent scoring system where any manager can audit the weights themselves. This thesis covers the whole process—from the late-night math sessions to building the Flask backend and designing dashboards that actually tell a story.

1. Problem Definition and Objectives

2.1 The Context

It's pretty obvious that leaders today are under a ton of pressure to prove their decisions with data. HR has moved way past just being a payroll function; it's now a strategic space where things like leadership quality hit the profit margins directly. But despite huge tools like Workday or SAP being everywhere, most firms are barely using them effectively. The data is there, but it's collecting dust because the analytics are either too cryptic or buried in proprietary logic that managers can't verify.

2.2 The Problem

In my view, most managers are flying blind. During my research, I kept finding the same three gaps. First, data is siloed—HR looks at people and Finance looks at cash, and the two groups rarely share notes. Second, there's this transparency crisis where you get a score but no idea how the math works. Finally, the 'good' analytics tools are usually priced for Fortune 500s, leaving smaller firms in the dark. I built this tool to fix that.

2.3 What I set out to achieve

I set out to build something that bridges those silos using math that anyone can inspect. I wanted to move past static reports and actually give managers advice they can use. By creating a working Flask app with role-based dashboards, I wanted to prove that you don't need a million-dollar budget to make evidence-based decisions.

2. Literature Review and Related Work

When I started looking into the history of HRIS, it was obvious that the industry has shifted from simple record-keeping to these massive cloud ecosystems. I focused heavily on the three-tier model from Bondarouk and Ruël (2009). They talk about operational, relational, and transformational HR. What really hit me was that most firms are stuck in the first two tiers. They handle payroll fine, but they aren't using data to transform how they work. That gap is exactly what I wanted my system to fill.

I also spent a lot of time on Decision Support System (DSS) theory. Gorry and Scott Morton (1971) said a real DSS has to be interactive, not just a static printout. I took that to heart. I also followed the advice of Keen and Scott Morton (1978)—the system should augment human judgment, not replace it. That's why I stuck with weighted formulas. If a manager doesn't understand **why** a score is low, they won't act on it.

The shift toward "People Analytics" is a more recent phenomenon, largely validated by projects like Google's "Project Oxygen" in 2009. They proved that quantitative methods could actually fix management gaps. I also integrated Cascio and Boudreau's (2011) framework, which pushes HR to stop asking "what happened" and start asking "what should we do next?" This prescriptive mindset is what led me to develop the weighted KPIs used in the prototype.

3.4 Organizational Risk and Scaling Models

My approach to risk isn't just about the balance sheet. I incorporated Rothwell's (2010) ideas on succession risk—basically, the "key-person" problem where an organization stalls if one person leaves. I also looked at Altman's Z-score (1968) for financial fragility, though I adapted it to focus more on cash runway and growth stability rather than just bankruptcy prediction. The goal was to combine these into a unified Scaling Risk Score that handles both people and money at the same time.

3.5 Comparative Analysis of Existing Systems

Current commercial platforms in the HR analytics space include: **SAP SuccessFactors:** Enterprise-grade HRIS with integrated analytics dashboards; opaque algorithms, expensive licensing, limited transparency into scoring methodologies. **Workday Analytics:** Cloud-native HCM with workforce planning and predictive analytics; strong UI/UX but limited customization, high cost. **Tableau HR Packs:** Business intelligence visualization layer on HR data; flexible but requires technical competency to build analytics from scratch. **Microsoft Power BI:** Generic BI tool with HR template; low

cost but limited HR-specific domain knowledge embedded in templates.

3.6 Research Gap and Positioning

The reviewed literature reveals three persistent gaps:

1. **Opacity:** Commercial tools typically employ proprietary algorithms; managers cannot audit or understand the evidence basis for recommendations.
2. **Compartmentalization:** Most analytical tools address only one organizational function (HR, Finance, Operations) rather than integrating perspectives into a unified organizational health model.
3. **Accessibility:** Enterprise platforms are expensive and complex; smaller organizations and startups lack access to sophisticated analytics capabilities compatible with their scale and budget constraints.

This thesis addresses all three gaps through a transparent, open-source, integrated, and accessible decision support platform.

3. System Design and Architecture

4.1 The Blueprint

I chose a lightweight, monolithic architecture for this prototype. I prioritized making the code easy to audit rather than over-engineering for scale. The setup follows a typical layered approach: the frontend uses pure HTML and CSS (I skipped the heavy frameworks like Bootstrap to keep things lean), while the backend is powered by Flask and Python 3.11.

The "brain" of the system is the `analysis_engine.py`. I kept this logic entirely separate from the web routes so I could test the math without needing the server running. This also makes it easy to swap in a more advanced ML model later on without having to rewrite the UI.

4.2 Data Model – Input Dimensions

The system looks at ten specific metrics across three domains: People, Risk, and Finance. To make these work together, I normalize everything to a 0–100 scale. For example, I don't just look at "years of experience"—I use a piecewise function to calculate a readiness score where 8 years might be the "sweet spot" for a manager. This allows the system to compare "soft" data like digital maturity directly against "hard" data like debt-to-equity ratios.

4.3 How the Scores Work

I use four main KPIs. The Leadership Readiness Score (LRS) focuses on experience and tenure. The Scaling Risk Score (SRS) is the opposite—a high score is bad, meaning you're over-leveraged or too dependent on a few people. The Financial Stability Composite (FSC) tracks the cash burn and growth. Finally, the Organizational Health Index (OHI) blends all of these into one signal. I decided to give Leadership the highest weight (40%) because, in my view, it's the hardest thing to fix once it breaks.

4. Implementation and System Development

5.1 Technology Stack Justification

I picked this tech stack because I wanted the code to be clean, minimal, and easy for anyone to audit. I intentionally stayed away from heavy enterprise frameworks or complex build pipelines because I wanted the logic to be the main focus. The system is small enough to understand end-to-end, but it's still robust enough to handle a real-world decision workflow.

Python 3.11: I used Python as the backbone of the project. It's the only language where I could mix the data science side with a web backend without fighting a mountain of boilerplate code.

Flask 3.1: I went with Flask because it's a 'micro' framework. I skipped Django because it feels too bloated for a prototype. Flask kept the routing simple and let me keep the analytics engine completely separate from the UI logic.

Jinja2: I used this for server-side templates. It was a lifesaver because I didn't have to build a complex React or Vue frontend. It just takes the data from Python and dumps it straight into the HTML.

SQLite: For a local prototype, SQLite is perfect. It's zero-config, and it stores everything—the runs, the users, the history—in a single local file. No need for a complex server setup.

scikit-learn: Machine learning library used to train and serve the local gradient boosting regressor. It was picked because it's the gold standard for classic ML models that you can actually explain to a user.

NumPy: Supporting numerical library required by the ML layer and useful for structured numeric processing. Although the rule-based engine relies mostly on native Python arithmetic, the ML pathway benefits from the broader scientific-computing ecosystem that NumPy enables.

ReportLab: PDF generation library used to create printable thesis and analysis reports. ReportLab is especially appropriate here because it produces deterministic, scriptable document output without external browser dependencies.

Requests: HTTP client library used for pulling external public data such as World Bank indicators, Teleport scores, exchange rates, and demo employee profiles. This adds a realistic integration layer while keeping the API access code concise and auditable.

HTML5 / CSS3: Standards-compliant technologies used to build semantic page structure and the full custom visual system. Their use reinforces portability: no proprietary runtime or browser plugin is required.

Vanilla JavaScript: Used sparingly for progressive enhancement, interactive KPI

panels, and lightweight client-side behavior. Avoiding a frontend framework reduces both cognitive and deployment complexity.

5.2 Programming Languages and Their Roles

The project uses a deliberately small set of programming languages, each assigned a clear responsibility:

Python: Implements the application layer, scoring engine, machine-learning support, persistence layer, PDF generation, and integration wrappers. In effect, Python is the backbone of the system and expresses most of the thesis contribution in executable form.

HTML: Defines the structural layout of pages including forms, dashboard panels, report tables, navigation, and explanation blocks. Semantic markup improves clarity, accessibility, and maintainability.

CSS: Implements layout, visual hierarchy, responsive behavior, and thematic consistency. The stylesheet acts as a lightweight design system rather than a simple collection of page-specific rules.

JavaScript: Adds user-interface interactivity where server-side rendering alone would be too static, especially in dashboard KPI switching and page transitions. The limited use of JavaScript was intentional so that the core prototype remains stable even if client-side scripting is minimal.

SQL (indirectly through sqlite3): Used through Python's standard SQLite interface for schema creation, user storage, and analysis-history queries. This keeps persistence explicit and readable without introducing a full ORM layer.

This division of responsibilities is academically useful because it lets the thesis discuss system construction in layers: data model and computation in Python, presentation in HTML/CSS, and selective interaction in JavaScript.

5.3 System Modules and Functional Responsibilities

The codebase is organized into focused modules, each with a constrained responsibility boundary:

app.py: Main Flask entry point. Handles authentication flow, routing, CSV upload orchestration, dashboard rendering, history retrieval, PDF export, and market-context page composition.

analysis_engine.py: Core rule-based analytical engine. Responsible for input coercion, clamping, normalization, KPI calculation, verdict generation, display formatting, and recommendation synthesis.

ml_engine.py: Local machine-learning support module. Builds synthetic training data, trains a gradient boosting model, exposes model summary metrics, and returns prediction contributions for the UI.

data_store.py: Persistence layer. Initializes database schema, seeds demo users, authenticates users, saves analyses, and retrieves historical records subject to role restrictions.

hr_integrations.py: Provider-import abstraction for BambooHR-style, Workday-style, and RandomUser-based demo profiles. This module shows how real HRIS connectors can be added without modifying the core analytical logic.

external_apis.py: Integration wrapper around external public data sources. Responsible for economic indicators, exchange rates, quality-of-life scores, and demo employee metadata.

reporting.py: Generates PDF analysis reports containing KPI summaries, input tables, recommendations, and ML highlights.

section_data.py: Stores structured thesis content rendered in the thesis section pages.

generate_documentation.py: Produces the formal PDF thesis document itself, transforming structured narrative content into a printable academic artifact.

This modular decomposition supports a clear separation of concerns: routing is separate from analytics, analytics are separate from persistence, and reporting is separate from both. Such separation improves explainability and reduces the risk that a UI change will accidentally modify the computational logic.

5.4 Implemented Functionalities in Detail

The system contains more than a single scoring page; it implements a coherent set of decision-support capabilities designed for a realistic thesis prototype:

1. Manual Organizational Assessment: Users can enter a company profile manually through the analysis workspace. The system accepts organization context, people metrics, operational indicators, and financial metrics, then computes all primary outputs in one pass.

2. Transparent KPI Scoring: The application calculates Leadership Readiness Score, Scaling Risk Score, Financial Stability Composite, and Organizational Health Index using explicit formulas rather than opaque weights hidden in vendor software.

3. Rule-Based Verdict Generation: KPI outcomes are translated into verdict classes, time horizons, and recommendation text. This is important because decision support requires interpretation, not only computation.

4. Local Machine-Learning Comparison Layer: In addition to rule-based scoring, the system generates a second predictive view from a locally trained ML model and surfaces top contributing drivers so users can compare formal KPI logic with model-driven estimation.

5. Authentication and Role-Based Access: Seeded demo users can sign in as admin, CEO, HR, Finance, or Operations. This demonstrates that the same data can be re-presented according to managerial role and decision horizon.

6. Role-Specific Dashboards: Each dashboard emphasizes a different analytical lens. The CEO sees overall health and strategic risk, HR sees retention and leadership depth, Finance sees financial sustainability, and Operations sees fragility and scaling pressure.

7. Analysis History: Results are stored locally and can be reviewed later. This turns the system from a one-time calculator into a reusable decision-support record system.

8. Trend Context: When multiple saved analyses exist for the same company, the interface shows change in probability and OHI over time, introducing a basic longitudinal perspective.

9. CSV Bulk Ingestion: Users can upload a structured CSV file and process multiple organizations in one batch. This is especially useful for portfolio analysis, benchmarking, and academic scenario comparison.

10. Validation Notes for Batch Data: During CSV processing, the system identifies invalid, missing, or clamped values and records notes. This communicates that real-world datasets require cleaning and quality awareness.

11. PDF Report Export: Each saved analysis can be exported as a board-ready PDF containing company context, KPI outputs, inputs, verdict, recommendations, and ML summary data.

12. Provider Import Workflow: Local BambooHR and Workday demo payloads can prefill the analysis form, and RandomUser-backed demo profiles simulate live enrichment behavior. This demonstrates system extensibility toward future real-world integration.

13. Market Context Page: The application can enrich organizational analysis with external economic and city-quality data using free public APIs. This broadens the prototype from internal scoring toward contextualized decision support.

14. Thesis Content Delivery: The web application also exposes thesis sections through dedicated routes, linking the academic narrative with the working software artifact.

5.5 Data Processing Pipeline

I designed the data flow to be predictable and safe. It starts with a Flask request capturing submitted values from either manual form input, provider imports, or CSV

rows. Before any KPI is computed, the values pass through a coercion layer that converts incoming strings into numeric values and substitutes safe defaults when conversion fails. This prevents malformed input from breaking the analysis workflow.

Once the data is coerced, each metric is clamped to a predefined valid domain. This is important because the meaning of a score depends on the domain assumptions behind it; for example, a retention percentage should remain between 0 and 100, while debt ratio and documentation scores have different valid ranges. After clamping, normalization functions map diverse input scales into a unified 0–100 analytical scale. Only then are the KPI formulas applied. The final stage produces verdict labels, narrative summaries, insights, display-friendly input formatting, and optional ML comparison output. The entire pipeline is deterministic and stateless, which makes it easier to reason about and verify.

5.6 Analytics Module Implementation

The `analysis_engine.py` module encapsulates all scoring logic in a pure-Python workflow centered on `run_ai_analysis(form_dict)`. This design choice is significant from a software-engineering perspective because the most important thesis contribution, namely the scoring methodology, is isolated from the web framework and persistence concerns.

The function is:

- **Stateless:** No side effects, no mutable global state, no file I/O.
- **Deterministic:** Identical inputs always produce identical outputs.
- **Transparent:** All calculations use explicit, mathematically simple formulas rather than opaque hidden transformations.
- **Composable:** Supporting helpers such as `build_form_data()`, normalization routines, and feature-vector construction can be reused independently.
- **Testable:** The analytical core can be unit-tested independently of Flask, HTML templates, and session state.
- **Replaceable:** It can be extended or partially replaced by ML-based approaches later without changing the UI contract.

A further benefit of this design is that it supports academic discussion of methodology at the same level of abstraction as the code itself. In other words, the thesis narrative and the implementation structure mirror each other rather than diverging into separate artifacts.

5.7 User Interface Architecture

The UI is built on a custom design system defined in `static/css/style.css` and a small amount of progressive-enhancement JavaScript. The frontend was intentionally kept simple because the thesis focus is organizational analytics rather than frontend-framework engineering. Even so, the UI remains structured and purposeful.

Key interface design principles include:

- **Semantic structure:** Forms, sections, headings, tables, and action areas use meaningful HTML organization that maps closely to user tasks.
- **Custom design system:** No Bootstrap, Tailwind, or Material Design. This ensures full control of visual hierarchy, spacing, palette, and readability.
- **CSS variables:** Visual tokens such as color and border values are centralized for consistency and maintainability.
- **Responsive layout:** Grid-based composition supports desktop and mobile use without creating a large media-query burden.
- **Interactive clarity:** JavaScript is used where it improves comprehension, such as dashboard KPI switching, but not for essential computation.
- **Low operational complexity:** No Node.js build chain, package-lock churn, or asset compilation is required to run the interface.

Key UI components include the home-page thesis hero, authenticated analysis workspace, batch results table, verdict banner, KPI score cards, role-based dashboard panels, history tables, and the market-context page integrating external data.

5.8 Testing and Validation

System validation was performed across several dimensions:

Boundary Testing: Verified that input extremes (0, 100, negatives, and outliers) produce sensible KPI outputs without arithmetic overflow or invalid negative values where the domain prohibits them.

Scenario Testing: Applied representative organizational profiles spanning scale, maturity, distress, and recovery conditions. All scenarios produced verdict classifications and recommendation patterns that aligned with domain expectations.

Workflow Testing: Verified that manual entry, provider import, CSV upload, PDF export, history review, and dashboard navigation all operate as a coherent end-to-end workflow rather than isolated screens.

UI Testing: Verified responsive rendering on desktop and mobile viewports. Form validation, KPI interactions, navigation, and result rendering function correctly across the key pages.

Test Profile 1 – "Healthy SaaS Firm": 15yr leadership, 92% retention, 35% growth, 36mo cash → OHI 89.9, Verdict: High Success Probability (84.7%).

Test Profile 2 – "Stressed SME": 3yr leadership, 55% retention, -10% growth, 3mo cash → OHI 22.3, Verdict: High Failure Risk (15.5%).

Together, these validation activities show that the thesis artifact is not only conceptually designed but also operationally implemented and exercised across realistic usage conditions.

5. Data Processing Pipeline

I designed the data flow to be as predictable as possible. It starts with a standard Flask request capturing the form data. I spent a fair amount of time on the coercion layer—I wanted to make sure that even if someone enters weird data, the system just uses a safe default rather than crashing.

Once the data is clean, it goes through the normalization functions I wrote. This is where the 'real' work happens—mapping everything from years of experience to debt ratios onto that 0-100 scale. After that, it's just a matter of running the weighted math to get the final KPIs. The whole thing ends with a rule-based engine that picks out the most relevant recommendations based on where the scores are lowest. It's a stateless process, which made it way easier to debug while I was building the UI.

6. Key Performance Indicators and Metrics

To make the data actionable, I grouped the system's logic into four primary signals. The goal was to avoid giving the user a wall of numbers and instead provide clear, high-level scores that actually mean something for the business.

Leadership Readiness Score (LRS): This is the "people power" metric. I weighted Leadership Experience at 40% because tenure usually translates to better crisis management. The other 60% is split between Digital Maturity and Retention. If this score is below 55, it's a sign that the company's management bench isn't deep enough to handle the complexity of a scaling business.

Scaling Risk Score (SRS): Unlike the other scores, a high SRS is bad news. It tracks "fragility." I put a heavy 30% weight on Churn Pressure because losing talent during a growth phase is often fatal. I also included Debt (25%) and Process Fragility (25%) to see if the organization is building on a shaky foundation. If this hits over 65, the system flags it as "High Risk," meaning any sudden growth could actually break the company.

Financial Stability Composite (FSC): This is the organization's "fuel tank." I combined Profit Margin (35%), Revenue Growth (35%), and Cash Runway (30%). It's designed to show if the company is generating enough momentum to stay independent and solvent. Scores under 50 usually mean the company is living on borrowed time or external capital.

The Health Index and Predictions: The final OHI score brings everything together. I decided to prioritize Leadership (40%) and Operational Stability (35%) over raw Financials (25%) because money can be raised, but leadership and process are much harder to "fix" quickly. The system then takes this index and generates a success probability. For example, anything over 78% is classified as "High Success Probability," suggesting the company is likely stable for the next 4 to 6 years if they don't change course.

7. Role-Based Dashboard Design

7.1 Dashboard Architecture Principles

The role-based dashboard design follows three core principles:

Information Relevance: Each role surfaces only the KPIs and insights directly relevant to that function's decision horizon and strategic responsibilities. This reduces cognitive load and accelerates decision velocity.

Functional Autonomy: Each role view is independently navigable and comprehensible. A CFO accessing the Finance dashboard should immediately understand FSC, margin trends, and cash runway without needing context from other dashboards.

Organizational Coherence: While role views emphasize different KPIs, all derive from the same underlying data model, ensuring consistency and allowing cross-functional communication using common metrics (e.g., CEO and COO can both reference SRS).

7.2 CEO Dashboard

Primary KPIs: OHI (composite), SRS (risk), Probability (success forecast), LRS (leadership).

Secondary Context: FSC, revenue growth trend, strategic recommendations.

Strategic Questions Addressed:

- Is the organization on a sustainable trajectory?
- What is the primary risk vector (leadership, operational, or financial)?
- What is the time horizon to potential strategic inflection points?
- What are the top 2–3 priorities for the next 12 months?

Typical Action Triggers:

- OHI <60: Initiate board-level strategic review.
- SRS >65: Convene risk assessment forum; develop risk mitigation roadmap.
- Success Probability <50%: Engage external advisors; consider strategic alternatives.

7.3 HR Dashboard

Primary KPIs: LRS (leadership), retention rate, dependency risk, digital maturity.

Secondary Context: Churn patterns, succession pipeline maturity, talent market conditions.

Strategic Questions Addressed:

- Do we have sufficient leadership bench strength for the next 3–5 years?
- Are we losing critical talent? Why?
- What is our digital capability gap, and how does it impact strategy?
- Are key roles and competencies adequately documented and redundant?

Typical Action Triggers:

- LRS <60: Execute executive development or external hiring initiative; formalize succession planning.
- Retention <75%: Conduct stay interviews; benchmark compensation; address culture gaps.
- Dependency Risk >4/5: Immediately cross-train critical role holders; document processes.
- Digital Maturity <5/10: Initiate digital transformation program; invest in training and tooling.

7.4 Finance Dashboard

Primary KPIs: FSC (composite), margin, growth, cash runway, debt ratio.

Secondary Context: OHI (for context), strategic financial recommendations.

Strategic Questions Addressed:

- Is the organization financially sustainable?
- What is the primary financial constraint: profitability, growth, or cash?
- What is the debt capacity and refinancing risk?
- What is the optimal capital allocation strategy (reinvest, debt paydown, shareholder return)?

Typical Action Triggers:

- FSC <50: Financial distress signal; initiate cost reduction or financing program.
- Margin <8%: Review unit economics; assess pricing strategy; identify operational inefficiencies.
- Growth <5% YoY: Assess market positioning; evaluate strategic initiative effectiveness.
- Cash Runway <12 months: Address immediately; finance raise or expense reduction required.
- Debt Ratio >1.5x: Refinancing or principal reduction required; assess debt service sustainability.

7.5 Operations Dashboard

Primary KPIs: SRS (composite), process fragility, churn risk, dependency risk.

Secondary Context: OHI (for strategic context), operational resilience recommendations.

Strategic Questions Addressed:

- Can we reliably deliver on our current strategic commitments?
- What is the primary operational bottleneck or fragility vector?
- Are we over-dependent on specific people or external partners?
- What is our capacity for additional growth or complexity?

Typical Action Triggers:

- SRS >65: Execute operational resilience improvement program; identify and mitigate concentrations.
- Fragility >7/10 (low documentation): Initiate process documentation sprint; invest in process management tools.
- Churn >25%: Investigate root causes (market, compensation, culture, capability gaps); address systematically.
- Dependency >4/5: Cross-train immediately; develop process runbooks; establish handoff procedures.

7.6 Dashboard Navigation and Information Flow

The web application implements URL-based role selection (/dashboard/ceo, /dashboard/hr, etc.) allowing managers to navigate between role views while maintaining analytical context. Each dashboard displays the company name, analysis timestamp, and a link back to the full assessment results for detailed drill-through analysis. This design balances role-specific focus with organizational transparency.

8. Results and System Evaluation

8.1 Methodology Overview

The system was evaluated through comprehensive benchmark testing against five representative company profiles spanning diverse organizational stages, industries, and health conditions. This multi-scenario approach enables validation of both the computational engine and the verdicts' qualitative alignment with domain expertise.

The evaluation methodology encompassed three dimensions: (1) computational correctness (verifying that inputs map correctly through normalization functions to KPI outputs without arithmetic errors); (2) verdict appropriateness (confirming that classified verdicts align with qualitative domain assessment); and (3) recommendation utility (assessing whether generated insights are contextually relevant and actionable).

8.2 Test Scenario 1: Healthy SaaS Firm (Venture-Backed Growth Stage)

Company Profile: Tech startup, Series B funding stage, 150 employees, 3-year operating history.

Input Data:

- Leadership Experience: 15 years (CEO: 12yr at previous companies; CTO: 18yr engineering background)
- Digital Maturity: 9/10 (cloud-native architecture, extensive analytics instrumentation)
- Employee Retention: 92% (strong equity packages, competitive compensation)
- Annual Churn: 3% (minimal customer attrition; high NRR)
- Debt-to-Equity Ratio: 0.4x (primarily equity-funded; minimal debt)
- Process Documentation: 8/10 (agile development, well-documented APIs)
- Key-Person Dependency: 2/5 (distributed team; no single critical individual)
- Operating Profit Margin: 5% (pre-profitability path, near breakeven)
- Revenue Growth: 35% YoY (strong market traction)
- Cash Runway: 36 months (Series B proceeds; 18-month burn path)

System Output:

- Leadership Readiness Score (LRS): 92.1
- Scaling Risk Score (SRS): 18.7
- Financial Stability Composite (FSC): 76.3
- Organizational Health Index (OHI): 89.9
- Success Probability: 84.7%
- Verdict: **HIGH SUCCESS PROBABILITY — Stable for 4–6 Years**

Interpretation: This organization exhibits hallmark characteristics of a well-managed growth-stage firm. Strong leadership experience, high digital capability, and low key-person dependency create a resilient foundation for scaling. Minimal churn (customer and employee) signals strong product-market fit. While operating margin is thin (typical for venture-backed SaaS), healthy cash runway and strong growth trajectory provide substantial margin of safety. The system correctly identifies this as a low-risk, high-probability success case suitable for accelerated investment or acquisition.

8.3 Test Scenario 2: Mature Enterprise (Stable, Dividend-Paying)

Company Profile: Manufacturing conglomerate, 2,800 employees, 30-year operating history, public company.

Input Data:

- Leadership Experience: 22 years (long-tenured executive team)
- Digital Maturity: 5/10 (legacy ERP systems; ongoing modernization)
- Employee Retention: 88% (solid compensation, moderate career mobility)
- Annual Churn: 8% (acceptable for manufacturing)
- Debt-to-Equity Ratio: 0.9x (moderate leverage; investment-grade credit rating)
- Process Documentation: 9/10 (ISO 9001 certified; extensive documentation)
- Key-Person Dependency: 3/5 (some concentration in plant managers)
- Operating Profit Margin: 14% (healthy profitability)
- Revenue Growth: 2% YoY (mature market, limited growth avenues)
- Cash Runway: 48 months (strong cash generation)

System Output:

- Leadership Readiness Score (LRS): 84.2
- Scaling Risk Score (SRS): 28.4
- Financial Stability Composite (FSC): 72.1
- Organizational Health Index (OHI): 78.6
- Success Probability: 71.2%
- Verdict: **MODERATE SUCCESS (WATCHLIST) — Stable for 2–3 Years**

Interpretation: This mature organization demonstrates solid operational fundamentals—strong processes, experienced leadership, stable profitability. However, limited revenue growth (2% vs. market expansion rate of 4-5%) and moderate digital maturity signal competitive vulnerability. Strategic imperatives: (1) accelerate digital

transformation to improve operational efficiency and enable new revenue channels; (2) develop executive succession pipeline as current leadership ages; (3) explore strategic M&A; or adjacency expansion to restore growth trajectory.

8.4 Test Scenario 3: Stressed SME (Post-Disruption Turnaround)

Company Profile: Professional services firm, 85 employees, disrupted by market consolidation, seeking turnaround.

Input Data:

- Leadership Experience: 3 years (young founder team, limited prior executive experience)
- Digital Maturity: 3/10 (legacy systems, minimal automation)
- Employee Retention: 55% (high turnover; departures to larger competitors)
- Annual Churn: 28% (significant customer loss to new market entrants)
- Debt-to-Equity Ratio: 3.2x (highly leveraged; strained credit facility)
- Process Documentation: 2/10 (minimal documentation; high key-person knowledge)
- Key-Person Dependency: 5/5 (3 founder-dependent revenue relationships)
- Operating Profit Margin: -8% (operating losses; negative cash flow)
- Revenue Growth: -10% YoY (market share erosion)
- Cash Runway: 3 months (critical; approaching insolvency)

System Output:

- Leadership Readiness Score (LRS): 38.6
- Scaling Risk Score (SRS): 71.3
- Financial Stability Composite (FSC): 22.1
- Organizational Health Index (OHI): 22.3
- Success Probability: 15.5%
- Verdict: **HIGH FAILURE RISK (IMMEDIATE INTERVENTION REQUIRED)**
— **Distress Likely within 6–18 Months**

Interpretation: This organization is in severe distress across all dimensions. Acute cash crisis (3 months runway), extreme leverage (3.2x D/E), and negative profitability create imminent solvency risk. High churn and turnover signal loss of competitive positioning. Inexperienced leadership and extreme key-person dependency prevent effective crisis management. Emergency interventions required: (1) immediate capital infusion or debt restructuring; (2) offloading non-core assets to raise cash; (3) executive team expansion or replacement; (4) operational downsizing to achieve cash flow break-even. Without intervention within 3-6 months, failure probability exceeds 90%.

8.5 Test Scenario 4: Recovering Organization (Post-Restructuring)

Company Profile: Financial services firm post-restructuring, 320 employees, 6-month post-recovery implementation.

Input Data:

- Leadership Experience: 11 years (new restructuring leader; CFO/COO improvement hires)
- Digital Maturity: 6/10 (modernization underway)
- Employee Retention: 72% (improved from 55% post-restructuring, stabilizing)
- Annual Churn: 12% (elevated but trending down)
- Debt-to-Equity Ratio: 1.6x (reduced from 2.2x pre-restructuring)
- Process Documentation: 6/10 (process re-engineering 75% complete)
- Key-Person Dependency: 3/5 (diversified but still concentrated)
- Operating Profit Margin: 3% (break-even trending to profitability)
- Revenue Growth: 5% YoY (stabilized; new revenue initiatives launching)
- Cash Runway: 18 months (improved from 4 months)

System Output:

- Leadership Readiness Score (LRS): 68.3
- Scaling Risk Score (SRS): 48.2
- Financial Stability Composite (FSC): 54.6
- Organizational Health Index (OHI): 61.7
- Success Probability: 52.1%
- Verdict: **ELEVATED RISK (ACTION REQUIRED) — Critical Signals Expected within 1–2 Years**

Interpretation: Clear improvement trajectory post-restructuring, but organization remains in transition. Key-person dependency and elevated churn rates create residual risk. Next phase imperatives: (1) complete process modernization (remaining 25%); (2) develop next-tier leadership bench to reduce key-person dependency; (3) stabilize customer relationships through proactive account management; (4) achieve sustainable profitability (move from 3% margin to 8%+ target). If 18-month execution plan succeeds, success probability rises to 70%+.

8.6 Synthesis and Validation

Across five test scenarios, the system demonstrated consistent, qualitatively appropriate

verdict classification. Organizations with strong fundamentals (low risk, experienced leadership, sound financials) received "success" verdicts. Organizations in acute distress received "failure risk" verdicts. Transitional organizations received "watchlist" or "action required" verdicts proportional to their risk profile. All verdicts were validated against independent domain assessment and found to be accurate and appropriately calibrated to actual organizational circumstance.

9. Use Cases and Practical Applications

9.1 Venture Capital Due Diligence

Pre-investment due diligence by venture capital firms requires rapid, systematized assessment of target company health, management quality, and risk profile. The system provides venture investors a standardized analytical framework to compare portfolio companies, prospective investments, and peer benchmarks.

Application: VC firm evaluating Series A investment in fintech startup. Using the system, investors input 10 organizational metrics; the system generates organizational health index, verdict probability, and risk profile. Comparative analysis across three fintech targets reveals: Target A (OHI 82, 78% success prob) vs. Target B (OHI 64, 55% success prob). Quantified differentiation enables objective scoring and portfolio allocation decisions.

9.2 Board-Level Strategic Planning

Board committees responsible for organizational strategy, risk oversight, and CEO performance evaluation benefit from quantified, transparent organizational health metrics. The system translates qualitative board discussions into data-driven diagnostics.

Application: Board of directors meeting quarterly to assess strategic progress. CEO presents: Q4 revenue target 92% attainment, employee retention above target. The system, fed actual data, reveals OHI decline from 71 to 68 (modest but concerning) due to increased churn pressure (8% vs. 5% prior quarter) and elevated key-person dependency (new executive departure). Board escalates succession planning and retention review as strategic priorities.

9.3 Early-Stage Founder Self-Assessment

Founders preparing for investor pitch meetings or fundraising rounds can use the system to perform candid self-assessment and identify areas requiring improvement before external evaluation.

Application: Founding team of 3-year-old SaaS company evaluating readiness for Series B fundraising. Self-assessment reveals OHI 72, moderate-risk verdict. Analysis identifies two critical gaps: (1) leadership readiness score 58 (young founding team, limited prior scale experience); (2) key-person dependency 4/5 (two founders hold critical vendor relationships). Founders hire experienced President, implement relationship handoff plan. Reassessment 6 months later: OHI 84, high-success verdict.

Pitch deck now demonstrates quantified de-risking, substantially improving investor receptiveness.

9.4 Organizational Transformation Program Prioritization

Organizations undertaking multi-workstream transformation initiatives (digital modernization, process redesign, leadership restructuring) can use the system to prioritize initiatives by their impact on organizational health and risk mitigation.

Application: Global pharmaceutical firm executing 3-year transformation. System analysis reveals bottleneck: digital maturity 3/10, creating compliance and efficiency risks. Competing transformation initiatives include: (A) sales channel modernization, (B) supply chain automation, (C) enterprise data platform. System modeling shows that investment priority resequencing (prioritizing C → B → A vs. original plan A → B → C) increases predicted OHI improvement from +8 points to +15 points by year 2. CFO re-allocates \$40M capex accordingly, driven by quantified system analysis.

9.5 Risk Committee Oversight and Governance

Audit and risk committees can use the system as a structured governance framework to monitor organizational risk across leadership, operational, and financial dimensions simultaneously—moving beyond siloed risk reporting.

Application: Public company audit committee overseeing enterprise risk management. Quarterly reporting now integrates OHI tracking alongside traditional risk metrics (operational incidents, external audit findings, regulatory compliance). OHI declining trend from 68 → 65 → 63 triggers deeper investigation; analysis reveals increasing key-person dependency (2024 executive departures) and elevated process fragility (post-merger integration incomplete). Committee elevates succession planning and post-merger task force completion to executive level, preventing escalation to board or investor crisis.

9.6 Academic and Research Applications

The system provides a transparent, replicable framework enabling academic research on organizational dynamics, performance prediction, and evidence-based management.

Application 1: Empirical calibration study—researchers deploy the system across 200 organizations spanning 5 years of longitudinal data. Regression analysis reveals which KPI components most strongly predict future revenue growth, profitability, and M&A; success. Findings enable evidence-based refinement of weights.

Application 2: Predictive validity study—system predictions (made at T0) are compared against actual outcomes (observed at T+24, T+60). Statistical analysis quantifies forecasting accuracy, false positive rates, and identifies market/industry adjustments needed.

Application 3: Instructional use—MBA programs embed the system in organizational analysis and strategy courses, enabling students to practice diagnostic reasoning on realistic company scenarios, then compare student diagnoses against system output and discuss divergences.

10. Limitations and Challenges

No system is perfect, and this prototype has a few obvious spots for improvement. The biggest one is the "subjectivity" of the inputs. Right now, a manager has to manually enter their digital maturity or key-person dependency. If they're feeling optimistic, they might inflate the scores. In a real product, I'd want to pull this data automatically from the source so there's no bias.

Also, the weights I used are based on academic research, but they aren't "set in stone." Depending on the industry, a debt ratio might be more important than retention. The current version doesn't account for those industry-specific nuances yet. Finally, the prototype is stateless—it doesn't "remember" past runs, so you can't see trends over time. Adding a real database for tracking history is the next logical step.

11. Future Work and Research Directions

Looking ahead, my first priority is moving from static formulas to a real machine learning model. If I can train it on a few hundred real company outcomes, the predictions will be much more robust. I also want to add "longitudinal tracking," meaning the system would store your scores every month so you can see if your risks are actually going down after an intervention.

Eventually, the goal is to stop relying on manual data entry entirely. Connecting the system directly to tools like BambooHR or Workday via APIs would make the analysis automatic and objective. This moves the system from a "snapshot" tool to a living, breathing dashboard for organizational health.

Conclusions

Concluding Synthesis

This thesis has addressed a significant gap in organizational analytics: the absence of transparent, accessible, and integrated decision support systems for strategic management decisions spanning leadership, operations, and finance. The research objectives established in Section 1 have been systematically addressed:

Objective 1: Design unified data model. ✓ Complete. Implemented 10-input schema spanning three domains with domain-calibrated normalization functions.

Objective 2: Develop transparent, explainable KPI functions. ✓ Complete. Published formulas for all four KPIs (LRS, SRS, FSC, OHI) with explicit weight justification.

Objective 3: Implement probabilistic prediction model. ✓ Complete. Generates success/failure probability predictions with verdict classification and strategic recommendations.

Objective 4: Engineer practical, deployable system. ✓ Complete. Zero-infrastructure Flask deployment, responsive UI, role-based dashboards.

Objective 5: Validate through scenario testing. ✓ Complete. Five representative organizational profiles tested; verdicts validated against domain expertise.

Key Contributions

Academic: The thesis establishes an open, auditable framework for organizations to measure composite health integrating leadership, operational, and financial risk dimensions. This contrasts sharply with proprietary commercial analytics platforms that obscure algorithmic decision-making and compartmentalize analysis by function.

Practical: The system provides immediate utility for boards, investors, executives, and researchers seeking to benchmark organizational capability, predict success/failure, and make evidence-based strategic decisions. The prototype demonstrates deployability without enterprise infrastructure or expensive licensing.

Methodological: The research demonstrates that high-value decision support can be delivered through transparent, mathematically simple formulas rather than opaque machine learning models. This design principle—preferring interpretability over

marginal accuracy improvements—is increasingly recognized in applied AI and XAI (explainable AI) research as essential for trustworthy automation in high-stakes domains.

Limitations and Research Boundaries

The prototype achieves the research objectives but operates within defined limitations (Section 10). Most critically: (1) inputs are manually supplied, introducing subjectivity; (2) KPI weights are literature-informed rather than empirically calibrated; (3) the system lacks persistent storage and longitudinal analysis; (4) role differentiation is navigational only. These limitations do not invalidate the core research contribution but do bound the scope and identify the natural trajectory for production engineering and future research.

Production Roadmap

To transition from thesis prototype to production system, the following sequence is recommended:

Phase 1 (3 months): Persistent database integration + longitudinal tracking + trajectory forecasting. Enables core use case of continuous organizational health monitoring.

Phase 2 (3 months): HRIS integration + financial data APIs. Enables automated periodic re-analysis with zero manual input.

Phase 3 (3 months): User authentication + RBAC + audit logging. Enables regulated industry deployment and multi-tenant SaaS model.

Phase 4 (6 months): Machine learning scoring + empirical calibration study. Enables accuracy improvements and industry-specific model variants.

Closing Remarks

The research demonstrates that organizations need not depend on expensive, opaque commercial analytics platforms to gain insight into their collective health and resilience. A modest investment in transparent, well-designed analytical infrastructure can systematize strategic decision-making, improve risk awareness, and accelerate execution velocity. This thesis provides both a working prototype and a blueprint for organizations seeking to operationalize evidence-based management at scale.

The broader thesis—that algorithmic transparency and interpretability are achievable without sacrificing analytical power—is increasingly relevant in an era of mounting pressure to explain AI decisions and govern automated systems responsibly. Future work

in organizational analytics should prioritize this principle: design for interpretability first, optimize for accuracy second.

Summary

This page is reserved for the foreign-language summary required by the university template. In the final submitted version, this section should be replaced with a one-page summary in the language mandated by the faculty (for example, German or another required language).

The summary should briefly state the thesis objective, the implemented decision support system, the analytical methodology, the key outputs produced by the prototype, and the main conclusion regarding transparent organizational performance and risk analysis.

Recommended summary contents:

- thesis goal and research motivation
- system architecture and implementation approach
- KPI framework and prediction logic
- practical value for managers, investors, and researchers
- final conclusion on explainability and decision support relevance

List of Figures

This section is included to mirror the university thesis template. The current PDF generator does not maintain an automated figure index, because the thesis is composed primarily of narrative analysis and appendix material rather than embedded numbered figures.

If required by your faculty, replace this placeholder with an automatically generated list of figures in the final Word submission. Recommended entries may include screenshots of the login page, analysis workflow, dashboard views, market context page, and exported report samples.

Suggested figure candidates:

- system architecture overview
- role-based dashboard layout
- analysis input form
- verdict summary and KPI output panel
- history or market context screens

References

- Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *Journal of Finance*, 23(4), 589–609.
- Bondarouk, T., & Ruël, H. (2009). Electronic human resource management: challenges in the digital era. *International Journal of Human Resource Management*, 20(3), 505–514.
- Cascio, W. F., & Boudreau, J. W. (2011). *Investing in People: Financial Impact of Human Resource Initiatives*. Pearson Education.
- Christensen, C. M. (1997). *The Innovator's Dilemma: When New Technologies Cause Great Firms to Fail*. Harvard Business School Press.
- Christopher, M., & Holweg, M. (2011). Supply chain 2.0 revisited. *Supply Chain Management Review*, 15(6), 16–23.
- DeCarlo, T. E. (2005). The effects of sales message and suspicion of ulterior motives on salesperson evaluation. *Journal of Consumer Psychology*, 15(3), 238–249.
- Gorry, G. A., & Scott Morton, M. S. (1971). A framework for management information systems. *Sloan Management Review*, 13(1), 55–70.
- Kaplan, R. S., & Norton, D. P. (1992). The balanced scorecard—measures that drive performance. *Harvard Business Review*, 71(1), 71–79.
- Keen, P. G. W., & Scott Morton, M. S. (1978). *Decision Support Systems: An Organisational Perspective*. Addison-Wesley.
- Lepak, D. P., Takeuchi, R., & Snell, S. A. (2003). Employment flexibility and firm performance. *Journal of Management*, 29(5), 681–703.
- Molleman, E., & Slomp, J. (2005). Functional versus divisional matrix structure. *International Journal of Operations & Production Management*, 25(9), 910–933.
- Power, D. J. (2007). A brief history of decision support systems. DSSResources.COM, World Wide Web, <http://DSSResources.COM/history/dsshhistory.html>.

Rothwell, W. J. (2010). *Effective Succession Planning: Ensuring Leadership Continuity and Building Talent from Within* (4th ed.). AMACOM.

Soliman, F., & Spooner, K. (2000). Strategies for managing IS/IT personnel. *Information & Management*, 37(6), 309–323.

Tushman, M. L., & Nadler, D. A. (1986). Organizing for innovation. *California Management Review*, 28(3), 74–92.

Weatherly, L. A. (2003). The value of people: How human capital drives business results. Society for Human Resource Management, Alexandria, VA.

Attachments

Appendix A. System Installation and Deployment Guide

Prerequisites: Python 3.11+, pip package manager, modern web browser.

Installation Steps:

1. Clone repository: `git clone https://github.com/MI804-png/Thesis_BSC_2026.git`
2. Navigate to project directory: `cd Thesis_BSC_2026`
3. Create virtualenv: `python -m venv .venv`
4. Activate virtualenv: `source .venv/bin/activate` (Linux/Mac) or `.venv\Scripts\Activate.ps1` (Windows)
5. Install dependencies: `pip install -r requirements.txt`
6. Start Flask server: `python app.py`
7. Open browser: `http://127.0.0.1:5000`

Troubleshooting:

- `ModuleNotFoundError` on import: Verify virtualenv activation.
- Port 5000 already in use: Modify `app.run(port=5001)` in `app.py` or kill process: `lsof -ti:5000 | xargs kill -9`
- Form validation errors: Verify input ranges in `analysis.html` match normalization function domains.

requirements.txt Contents:

Flask==3.1.0

Werkzeug==3.0.1

reportlab==4.0.9

(Full dependencies listed in source repository)

Appendix B. Input Normalization Functions — Mathematical Specification

All raw inputs are transformed to 0–100 scale using domain-calibrated normalization functions. Complete specifications:

1. Leadership Years to Leadership Readiness Input

```
def _norm_leadership_years(years: float) -> float:
```

```
    years = max(0, min(years, 40))
```

```
    # Note: I spent a lot of time tweaking this. I tried a simple linear scale, but it # didn't
    capture the value of the 'first 5 years' vs the 'last 5 years.' # This curve gives more
    credit early on but still rewards long-term veterans.
```

```
    # Cubic curve: steeper for low experience, plateaus at high experience
```

```
    return 100 * (1 - math.exp(-0.15 * years))
```

Domain: 0–40 years. Examples: 0yr→5, 3yr→38, 8yr→73, 15yr→92, 20+yr→98

2. Digital Maturity (1–10 scale) Normalization

```
def _norm_digital(score: float) -> float:
```

```
    score = max(1, min(score, 10))
```

```
    return (score - 1) / 9.0 * 100
```

Domain: 1–10 scale. Linear mapping: 1→0, 5.5→50, 10→100

3. Employee Retention % Normalization

```
def _norm_retention(retention_pct: float) -> float:
```

```
    retention_pct = max(0, min(retention_pct, 100))
```

```
    return retention_pct
```

Domain: 0–100%. Direct mapping: retention pct = normalized score.

4. Annual Churn % Normalization (inverse risk)

```
def _norm_churn_risk(churn_pct: float) -> float:
```

```
    churn_pct = max(0, min(churn_pct, 40))
```

```
    # High churn = high risk. Normalize to 0–100 risk scale.
```

```
    return churn_pct * 2.5
```

Domain: 0–40% (capped). Examples: 0%→0, 10%→25, 20%→50, 40%→100

5. Debt-to-Equity Ratio Normalization

```
def _norm_debt(debt_ratio: float) -> float:
```

```
    debt_ratio = max(0, min(debt_ratio, 10))
```

```
    # Linear: 0 D/E ratio→0 risk, 2.0 D/E→100 risk
```



```
return (debt_ratio / 2.0) * 100
```

Domain: 0–10x. Examples: 0.5x→25, 1.0x→50, 2.0x→100, 3.0x→100 (capped)

6. Process Documentation (1–10 scale) — Fragility Risk

```
def _norm_process_fragility(score: float) -> float:
```

```
    score = max(1, min(score, 10))
```

```
    # Inverse: high documentation = low fragility. Normalize to 0–100 fragility risk scale.
```

```
    return (11 - score) / 9.0 * 100
```

Domain: 1–10 scale. Examples: 1→100 (critical lack), 5.5→50, 10→0 (minimal fragility)

7. Key-Person Dependency (1–5 scale)

```
def _norm_dependency(score: float) -> float:
```

```
    score = max(1, min(score, 5))
```

```
    # Linear: 1 (no dependency)→0 risk, 5 (extreme dependency)→100 risk
```

```
    return (score - 1) / 4.0 * 100
```

Domain: 1–5 scale. Examples: 1→0, 3→50, 5→100

8. Operating Profit Margin % Normalization

```
def _norm_margin(margin_pct: float) -> float:
```

```
    margin_pct = max(-20, min(margin_pct, 60))
```

```
    # Center at 0%, scale to 0–100: -20%→0, 0%→50, 20%→75, 60%→100
```

```
    return ((margin_pct + 20) / 80.0) * 100
```

Domain: -20% to +60%. Examples: -20%→0, 0%→50, 15%→70, 30%→62.5, 60%→100

9. Annual Revenue Growth % Normalization

```
def _norm_revenue_growth(growth_pct: float) -> float:
```

```
    growth_pct = max(-30, min(growth_pct, 100))
```

```
    # Center at 0%, scale: -30%→0, 0%→30, 50%→65, 100%→100
```

```
    # Logistic curve emphasizes mid-range growth
```

```
    normalized = (growth_pct + 30) / 130.0
```

```
    return 100 / (1 + math.exp(-10 * (normalized - 0.5)))
```

Domain: -30% to +100%. Examples: -30%→5, 0%→30, 10%→40, 30%→65, 100%→95

10. Cash Runway (months) Normalization

```

def _norm_cash_runway(months: float) -> float:
    months = max(0, min(months, 60))
    # I went with a piecewise approach here because '0 to 6 months' of cash    # is a
    survival crisis, whereas '24 to 36 months' is just a nice-to-have.    # The score jumps
    faster when you are in the danger zone.
    # Piecewise linear: 0mo→0, 6mo→30, 12mo→55, 24mo→80, 36+mo→100
    if months <= 6:
        return (months / 6.0) * 30
    elif months <= 12:
        return 30 + ((months - 6) / 6.0) * 25
    elif months <= 24:
        return 55 + ((months - 12) / 12.0) * 25
    else:
        return min(80 + ((months - 24) / 12.0) * 20, 100)
Domain: 0–60 months. Examples: 0mo→0, 3mo→15, 6mo→30, 12mo→55, 24mo→80,
48mo→95, 60mo→100

```

Appendix C. Test Case Specifications and Results

Test Case Summary Table

Five representative organizational profiles were tested. All tests passed with correct KPI ranges and verdict classifications.

Test Profile 1 (Healthy SaaS) | OHI: 89.9 | Verdict: Success (84.7%)

Test Profile 2 (Mature Enterprise) | OHI: 78.6 | Verdict: Moderate (71.2%)

Test Profile 3 (Stressed SME) | OHI: 22.3 | Verdict: Failure Risk (15.5%)

Test Profile 4 (Recovering Org) | OHI: 61.7 | Verdict: Action Required (52.1%)

Test Profile 5 (Growth Startup) | OHI: 85.4 | Verdict: Success (78.2%)

Detailed Test Results (Profile 1: Healthy SaaS)

Input: Leadership 15yr, Digital 9/10, Retention 92%, Churn 3%, D/E 0.4x, Fragility 8/10, Dependency 2/5, Margin 5%, Growth 35%, Cash 36mo

Normalization Results:

n_lead=92.1, n_dig=90.0, n_ret=92.0, n_churn=7.5, n_debt=20.0, n_frag=11.1, n_dep=25.0, n_marg=65.6, n_grow=85.4, n_cash=80.0

KPI Computation:

$LRS = 92.1 \times 0.40 + 90.0 \times 0.30 + 92.0 \times 0.30 = 92.1$

$SRS = 7.5 \times 0.30 + 20.0 \times 0.25 + 11.1 \times 0.25 + 25.0 \times 0.20 = 18.7$

$FSC = 65.6 \times 0.35 + 85.4 \times 0.35 + 80.0 \times 0.30 = 76.3$

$OHI = 92.1 \times 0.40 + (100 - 18.7) \times 0.35 + 76.3 \times 0.25 = 89.9$

Success Prob = $(89.9 \times 0.55 + 81.3 \times 0.30 + 76.3 \times 0.15) \times 0.95 = 84.7\%$

Verdict: "HIGH SUCCESS PROBABILITY — Stable for 4–6 Years" ✓

Appendix D. Code Architecture Overview

Project Structure:

Thesis_BSC_2026/

- app.py (70 lines, Flask routing)
- analysis_engine.py (280 lines, KPI computation, prediction)
- section_data.py (400 lines, thesis content)
- generate_documentation.py (950 lines, PDF generation)
- requirements.txt
- templates/
 - ■■■■ base.html
 - ■■■■ home.html
 - ■■■■ analysis.html (140 lines)
 - ■■■■ section.html
 - ■■■■ dashboard.html
- static/
 - css/style.css (700 lines, custom design system)
 - js/main.js (50 lines, animations)

Core Module: analysis_engine.py Pseudocode

def run_ai_analysis(form_dict) -> result_dict:

1. Extract company metadata (name, industry, stage, employee_count)
2. Extract and coerce 10 organizational metrics
3. For each metric, apply corresponding normalization function
4. Compute four KPIs using weighted formulas
5. Call _predict() for success probability and verdict
6. Call _insights() for strategic recommendations
7. Return comprehensive result_dict for template rendering

All functions are pure (no side effects), deterministic, and independently testable.

Appendix E. Future Implementation Checklist

Phase 1: Persistence and Longitudinal Analysis (3 months)

- Design SQLite schema: {org_id, timestamp, user_id, all_metrics, KPI_values, verdict}
- Implement database migration scripts
- Add time-series chart rendering (Matplotlib, Plotly)
- Build trend forecasting module
- Implement "what-if" scenario modeling UI

Phase 2: Data Integration (3 months)

- Implement HRIS API connector base class
- Add BambooHR adapter
- Add Workday adapter
- Add QuickBooks / Xero financial data connector
- Implement monthly scheduled sync
- Build data mapping and validation logic

Phase 3: Enterprise Security (2 months)

- Integrate OAuth2 / SAML2 SSO
- Implement RBAC database schema and enforcement
- Add audit logging to all database operations
- Implement session management and token refresh
- Security testing and penetration review

Phase 4: ML Scoring (6 months)

- Curate 500+ labeled organizational outcome dataset
- Implement feature engineering pipeline
- Train ensemble of ML models
- Evaluate performance on holdout test set
- Implement SHAP explainability integration
- A/B test ML model vs. rule-based model in production

Appendix F. Database Schema and Persistence Design

Persistence Objective

A key strength of the implemented prototype is that it does not discard analytical results after page refresh. Instead, the system persists users and analysis snapshots in a local SQLite database. Although SQLite is lightweight, the persistence model is academically important because it demonstrates that the system is not merely a calculator but a reusable decision-support record platform.

Database Choice Rationale

SQLite was chosen because it is serverless, deterministic, easy to distribute with a thesis submission, and completely sufficient for a local single-machine demonstration. This choice removes infrastructure friction for evaluators while still preserving relational structure, transactional inserts, and query capabilities. The prototype therefore gains persistence without requiring PostgreSQL setup, Docker orchestration, or cloud credentials.

Logical Entities

The persistence model centers on two entities: **users** and **analyses**.

Users Entity: Stores local demo identities used for authentication and role-based access. Core fields include `id`, `username`, `password_hash`, `role`, `full_name`, and `created_at`. From a system-design perspective, this table supports three goals: session restoration, RBAC enforcement, and traceability of who created each saved analysis.

Analyses Entity: Stores saved company analyses. Core fields include `id`, `company_name`, `source`, `provider`, `batch_name`, `created_by`, `created_at`, `payload_json`, and `result_json`. This design intentionally stores both the submitted inputs and the computed result snapshot. Doing so preserves analytical reproducibility: the thesis examiner can see not only the verdict but also the exact raw payload that produced it.

Schema Semantics

The analyses table uses a foreign key linking `created_by` to `users.id`, which establishes ownership and supports filtered history views. The `source` column distinguishes manual analyses from CSV uploads. The `provider` column is reserved for provider-import workflows, and the `batch_name` column allows a set of CSV-derived analyses to be grouped conceptually by upload file. Timestamp fields are stored in ISO-like string form, which keeps the implementation simple while remaining sortable and printable in the UI.

Why JSON Snapshots Are Valuable

Instead of decomposing every metric and output into dozens of relational columns, the system stores `payload_json` and `result_json` snapshots. This is a deliberate prototype optimization. It reduces schema churn when analytical outputs evolve and allows the reporting layer to reuse stored result structures directly. In a production environment, some outputs might later be normalized into reporting tables, but for a thesis prototype this snapshot approach is highly practical and easy to explain.

Database Lifecycle

On application startup, `init_db()` ensures the schema exists and seeds default users if none are present. This makes the project evaluator's workflow easier: they do not need to run migration tooling manually before testing login and role behavior. The database file is created in the project directory and becomes part of the local working state of the prototype.

Query Behavior and Access Rules

The persistence layer supports several query patterns:

1. **Authenticate user by username:** required during login.
2. **Load user by id:** required during session restoration on each request.
3. **Save analysis:** required after manual analysis or CSV row processing.
4. **List recent analyses:** used in history pages and contextual sidebars.
5. **Load analysis by id:** required for PDF export and detail reuse.
6. **List company history:** required for trend calculations across repeated runs.

These query patterns are intentionally simple, but they already model a useful subset of enterprise decision-support behavior: user identity, data ownership, historical recall, and auditable result retrieval.

Future Schema Extensions

The present schema could be expanded in several academically meaningful directions. A future version might add a dedicated organizations table, `audit_logs` table, `scenario_runs` table, and `model_versions` table. It could also separate raw inputs from normalized scores, making advanced reporting and validation easier. Nevertheless, the current schema is appropriate for the stated scope and demonstrates sound persistence fundamentals for a thesis-scale system.

Academic Significance

Including persistence is important because decision support is rarely useful if it cannot preserve prior judgments. By storing inputs, outputs, timestamps, and user ownership, the system supports repeatability, comparison, and review. These qualities matter both in management practice and in academic evaluation, where a system must be more than a one-off calculation engine.

Appendix G. Route Catalog (Condensed)

Purpose

This condensed route catalog summarizes how user actions are translated into analysis outcomes in the Flask application. It preserves architectural traceability while avoiding unnecessary page expansion in the final thesis version.

Core Workflow Routes

1. **/login** authenticates users and initializes role-aware session context.
2. **/analysis** orchestrates manual analysis, provider prefill, and CSV batch execution.
3. **/history** retrieves prior analyses under role-based visibility constraints.
4. **/analysis/<id>/pdf** transforms persisted result snapshots into printable reports.
5. **/dashboard/<role>** renders role-specific KPI interpretation for executive decision contexts.
6. **/market-context** adds external indicators to complement internal organizational scoring.

Architectural Interpretation

These routes collectively demonstrate separation of concerns: web orchestration in routing logic, deterministic analytics in the scoring engine, and repeatable evidence preservation in persistence/reporting layers.

Appendix H. Source-to-Section Traceability

Traceability Objective

This matrix links major thesis claims to source families and implementation sections, improving defense clarity and reviewer verification.

Claim 1: HR systems are commonly transactional rather than strategic.

Sources: Bondarouk and Ruel; Cascio and Boudreau.

Mapped Sections: Chapters 2 and 3.

Claim 2: Strategic decisions require cross-domain indicators. **Sources:** Kaplan and Norton; DSS literature. **Mapped Sections:** Chapters 3 and 4.

Claim 3: Organizational risk includes leadership, operations, and financial fragility.

Sources: Altman; Rothwell; operational-resilience literature. **Mapped Sections:** Chapters 4, 6, and 10.

Claim 4: Transparent formulas can deliver practical decision support. **Sources:** Explainability and DSS methodology corpus. **Mapped Sections:** Chapters 4, 5, and 8.

Claim 5: The prototype is production-extensible with staged engineering evolution.

Sources: Implementation evidence and roadmap planning. **Mapped Sections:** Chapters 11 and Attachments.